

OBSERVACIONES DE LA PRÁCTICA

- 1. Nicolás Fajardo Aguirre, 202413083, n.fajardoa@uniandes.edu.co
- 2. Christian Bravo, 202322012, ca.bravom@uniandes.edu.co
- 3. Dennys Lozano, 202413859, da.lozanom1@uniandes.edu.co

Máquina 1	
Procesador	12th Gen Intel(R) Core(TM) i5-1235U
Memoria RAM (GB)	12 GB
Sistema Operativo	Windows 11 Home Single Language 64 bits

Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.

Resultados

Carga de Catálogo LINEAR PROBING

Factor de Carga (PROBING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @LP [ms]
0.1	467 473.71	5 220.95
0.5	379 216.05	4 574.15
0.7	372 786.26	4 553.14
0.9	368 142.48	4 987.85

Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING

Carga de Catálogo SEPARATE CHAINING

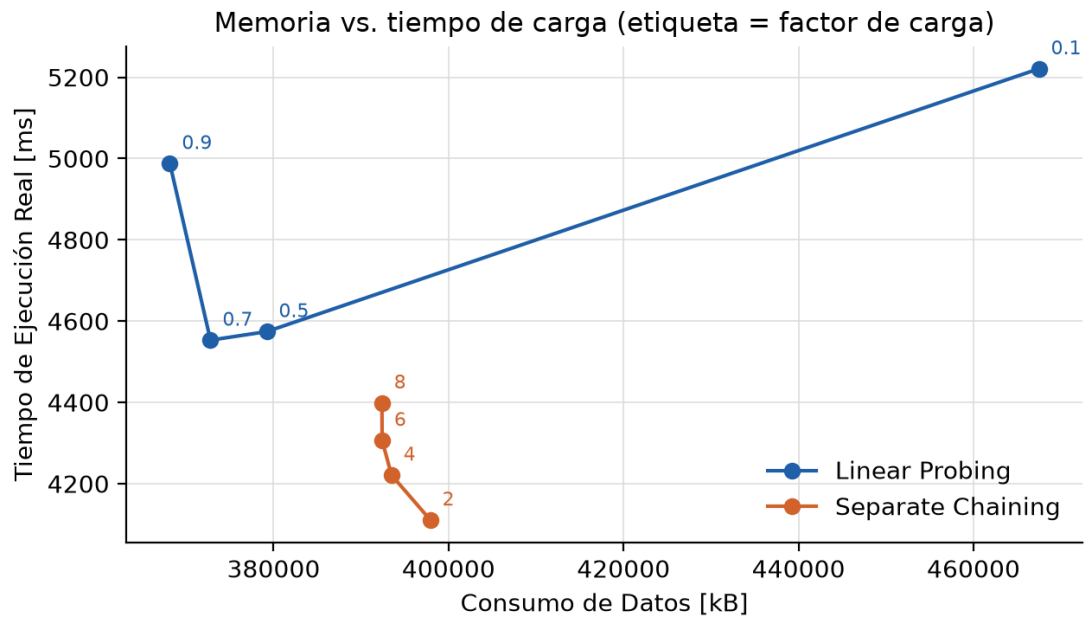
Factor de Carga (CHAINING)	Consumo de Datos [kB]	Tiempo de Ejecución Real @SC [ms]
2.00	397 909.79	4 110.82
4.00	393 473.35	4 221.24
6.00	392 405.39	4 306.62
8.00	392 380.81	4 398.80

Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING

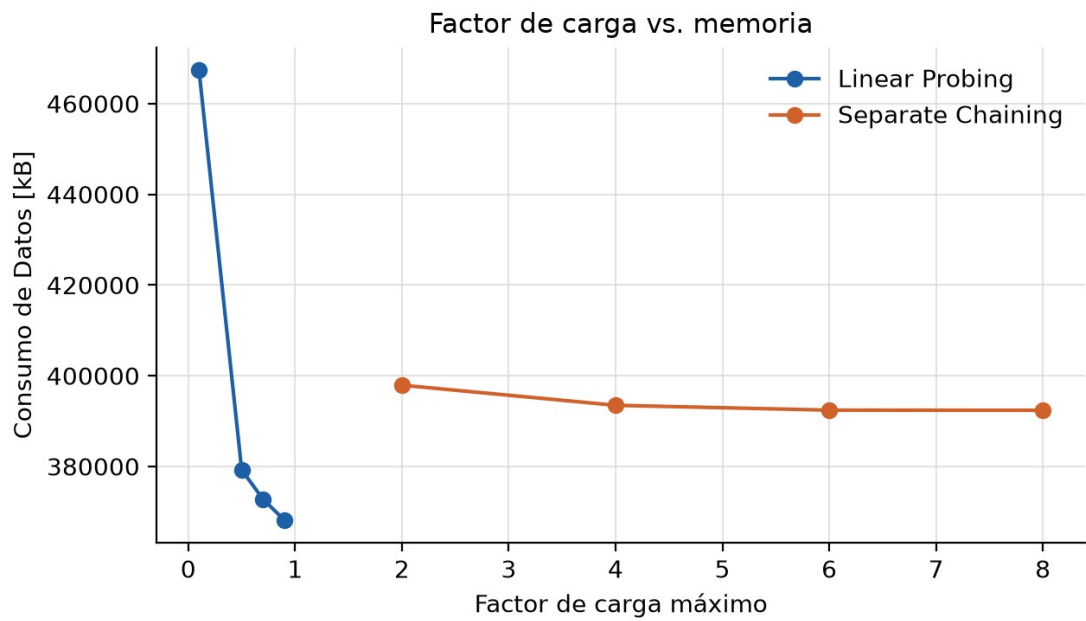
Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento.

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING



- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING



Preguntas de análisis

1. ¿Por qué en la función `getTime()` se utiliza `time.perf_counter()` en vez de otras funciones como `time.process_time()`?

RTA: Porque `time.perf_counter()` mide el tiempo real transcurrido (reloj de pared) con la mayor resolución disponible en el sistema, incluyendo los momentos en que el proceso está esperando, por ejemplo durante la lectura de los archivos CSV. `time.process_time()`, en cambio, solo cuenta el tiempo de CPU consumido por el proceso y excluye las esperas de entrada/salida y el tiempo suspendido, así que subestimaría lo que realmente tarda la carga del catálogo. Como la columna pedida es el Tiempo de Ejecución Real, la función adecuada es `perf_counter()`.

2. ¿Por qué son importantes las funciones `start()` y `stop()` de la librería `tracemalloc`?

RTA: `tracemalloc.start()` activa el rastreo de las asignaciones de memoria de Python: sin llamarla, las muestras tomadas con `take_snapshot()` no contienen información y no se puede calcular la diferencia. `tracemalloc.stop()` detiene el rastreo y libera la memoria que la propia librería usa para guardar las trazas. Juntas delimitan exactamente la región de código que se quiere medir y evitan que el costo adicional del rastreo afecte al resto del programa.

3. ¿Por qué no se puede medir paralelamente el **uso de memoria** y el **tiempo de ejecución** de las operaciones?

RTA: Porque `tracemalloc` intercepta cada asignación de memoria y guarda su traza, lo que hace mucho más lenta la ejecución; además, tomar las muestras (snapshots) también consume tiempo. En nuestras pruebas, la misma carga tardó entre 12 586 y 16 071 ms con `tracemalloc` activo, frente a 4 111–5 221 ms sin él (aproximadamente 3 veces más). Por eso el tiempo medido junto con la memoria no representa el tiempo real, y las dos mediciones se tomaron en ejecuciones separadas: el tiempo con el rastreo apagado y la memoria en otra corrida.

4. Dado el número de elementos de los archivos (`large`), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?

RTA: El factor de carga real es $\alpha = N/M$ (elementos sobre capacidad de la tabla). Los archivos `large` tienen 10 000 libros, 5 841 autores y 34 252 tags, y los índices se crearon con capacidad inicial para 10 000, 6 000 y 34 500 elementos, de modo que ninguno necesitó rehash y el factor real queda apenas por debajo del máximo configurado. Por ejemplo, para el índice de tags: con Linear Probing 0.099, 0.496, 0.695 y 0.893 (máximos 0.1, 0.5, 0.7 y 0.9); con Separate Chaining 1.98, 3.97, 5.93 y 7.92 (máximos 2, 4, 6 y 8). En Linear Probing el factor siempre debe ser menor que 1, porque cada posición guarda una sola pareja, y un valor cercano a 0.5 es el recomendado; en Separate Chaining puede ser mayor que 1, porque cada posición guarda una lista, y un valor cercano a 4 es razonable.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?

RTA: Los cambios son pequeños (menos del 15 % entre el mejor y el peor caso de cada esquema), porque la carga está dominada por la lectura de los archivos y por cerca de un millón de inserciones de `book_tags`. En Linear Probing el tiempo tiene forma de U: 5 221 ms con 0.1, 4 574 ms con 0.5, 4 553 ms con 0.7 y 4 988 ms con 0.9. Con 0.1 se pierde tiempo creando tablas muy grandes y casi vacías, y con 0.9 aparecen agrupamientos (clusters): el promedio de posiciones revisadas por llave sube de 1.5 (0.5) a 5.2 (0.9), con secuencias de hasta 531 posiciones. En Separate Chaining el tiempo crece de forma suave y sostenida con el factor: 4 111, 4 221, 4 307 y 4 399 ms para 2, 4, 6 y 8, porque las listas de cada posición son más largas (en promedio 2.0, 3.0, 3.9 y 5.0 nodos revisados por llave). Las diferencias entre factores vecinos son del mismo orden que la variación entre repeticiones (desviación de 90 a 330 ms), así que solo la tendencia general es concluyente.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?

RTA: En Linear Probing la memoria disminuye al aumentar el factor de carga: 467 474 kB (457 MB) con 0.1, 379 216 kB con 0.5, 372 786 kB con 0.7 y 368 142 kB (360 MB) con 0.9, una reducción del 21 %. La capacidad de la tabla es N/α , así que un factor bajo reserva muchas posiciones vacías; el salto más grande está entre 0.1 y 0.5, y de ahí en adelante el ahorro es marginal. En Separate Chaining la memoria casi no cambia: de 397 910 kB con 2 a 392 381 kB con 8 (1.4 % menos), porque cada pareja ocupa su propio nodo sin importar el factor y lo único que varía es el número de listas vacías, que es pequeño. En ambos casos la mayor parte de la memoria corresponde a los datos cargados (libros y asociaciones `book_tags`), no a las tablas.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

RTA: Sí se perciben. Separate Chaining fue más rápido que Linear Probing en todas las configuraciones: entre 4 111 y 4 399 ms frente a 4 553–5 221 ms, en promedio un 12 % menos. Comparando los factores recomendados, Separate Chaining con 4 tardó 4 221 ms y Linear Probing con 0.5 tardó 4 574 ms. La razón es que en Separate Chaining una colisión solo afecta a las llaves de la misma posición y sus tablas son mucho más pequeñas, por lo que se crean más rápido; en Linear Probing las colisiones se propagan a las posiciones vecinas y forman clusters, y además hay que crear tablas con muchas más posiciones. Separate Chaining también es más estable: su tiempo varía un 7 % entre factores, contra un 15 % en Linear Probing.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.

RTA: Sí se perciben. Con los factores habituales Separate Chaining consume más memoria: 393 473 kB con factor 4 frente a 379 216 kB de Linear Probing con 0.5 (3.8 % más), y se mantiene por encima de Linear Probing con 0.7 y 0.9. Esto se debe a que en Separate Chaining cada pareja necesita, además de la entrada llave-valor, un nodo de lista encadenada con su referencia al siguiente, y cada posición de la tabla guarda una lista aunque esté vacía. Linear Probing guarda las entradas directamente en el

arreglo. La excepción es Linear Probing con 0.1, que fue la configuración que más memoria usó (467 474 kB), porque reserva diez posiciones por cada elemento. En resumen, la memoria de Linear Probing depende mucho del factor de carga y la de Separate Chaining casi nada.